

From Curiosity to Creation: Build Your First AI Project & Research Portfolio

Prof. Ping Chen Inal Mashukov

Department of Computer Science
University of Massachusetts Boston

DAY 2 - Inside the Black Box: Mechanistic Interpretability

Summer 2026

The "Black Box" Problem in AI

The Interpretability Crisis

We know *how* to build and train massive neural networks, but we do not fully understand *what* they are doing inside.

- We put data in (X).
- We get answers out (Y).
- The middle is billions of floating-point numbers interacting in high-dimensional space.

Why is this dangerous?

If we deploy an AI to drive a car, diagnose a patient, or approve a loan, and it makes a catastrophic error, we cannot currently look at the code to find out *why*. We have to reverse-engineer its "brain."

Before 2017: The Problem with Time

Recurrent Neural Networks (RNNs)

Before modern LLMs, AI read text exactly like humans do: one word at a time, left to right.

- *The problem:* If a paragraph is 500 words long, by the time the AI reaches word 500, it has forgotten word 1.
- It was fundamentally impossible to parallelize training across GPUs because word 4 required word 3 to be processed first.

The 2017 Breakthrough

Google researchers published a paper titled "*Attention Is All You Need.*" It introduced the **Transformer**. It processes every word in a document simultaneously.

Step 1: If Not Sequence, Then What?

If the Transformer processes the entire sentence at once, how does it know the order of the words?

"The dog bit the man" vs. *"The man bit the dog"*
are the exact same words, but radically different meanings.

Positional Encoding

We must explicitly inject a mathematical "time-stamp" into the word embeddings before they enter the network.

$$\text{Input} = \text{Word Embedding} + \text{Positional Vector}$$

The Math of Positional Encoding

Transformers use intersecting sine and cosine waves of different frequencies to create a unique coordinate for every position in a sequence.

The Formulas

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos : The position of the word in the sentence (0, 1, 2...).
- i : The dimension within the embedding vector.
- d_{model} : The total size of the embedding (e.g., 768).

Hands-On Practice 1: Build a Positional Encoding

Mission: You are the Transformer. Give each word a mathematical “address.”
Suppose our tiny Transformer uses only two dimensions:

$$PE(pos) = [\sin(pos), \cos(pos)].$$

For the sentence: The cat sat
use positions: The = 0, cat = 1, sat = 2.

Your Challenge

Using a calculator or ChatGPT, calculate the two numbers for each word. Round to 2 decimal places.

$$PE(0), \quad PE(1), \quad PE(2)$$

Then answer:

How can the model tell “cat” and “sat” apart?

Bonus: What happens if two words have the same word embedding but different positions?

Practice 1: Answer — Positional Encoding

For

$$PE(pos) = [\sin(pos), \cos(pos)],$$

we obtain:

Word	Position	Positional Vector
The	0	(0.00, 1.00)
cat	1	(0.84, 0.54)
sat	2	(0.91, -0.42)

Therefore, even if “cat” and “sat” have completely different or even identical semantic embeddings, their positional vectors are different.

Big Idea

The Transformer processes all tokens in parallel, so it needs an additional mathematical signal telling it:

“Where am I in the sentence?”

Input = Word Embedding + Positional Vector

Step 2: The Self-Attention Mechanism

What is Attention?

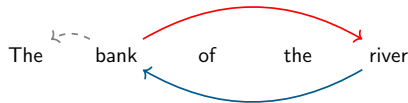
As the model processes a word, it "looks around" at every other word in the sentence to gather context.

Context Resolution

"The **bank** of the river."

"The **bank** approved the loan."

The embedding for "bank" starts the same. But through Self-Attention, "bank" pulls meaning from "river" or "loan" to update its own vector.



The Database Analogy: Queries, Keys, and Values

How does a word decide which other words to pay attention to? It uses a mathematical retrieval system.

- **Query (Q):** What I am looking for. (e.g., "I am an adjective looking for my noun.")
- **Key (K):** What I possess. (e.g., "I am a noun.")
- **Value (V):** My actual semantic content. (e.g., "I mean 'river'.")

Every word in the sentence generates its own Q , K , and V vectors by multiplying its embedding by three learned weight matrices (W_Q, W_K, W_V).

The Core Equation of Modern AI

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Let's break this down mechanistically:

- ① QK^T : We take the dot product of every Query with every Key. This generates a massive "Relevance Score" grid. High dot product = High relevance.
- ② $\sqrt{d_k}$: We scale the scores down so the gradients don't explode during training.
- ③ softmax: We convert the scores into percentages that sum to 1.0.
- ④ V : We multiply the percentages by the Values to get the final updated representation of the word.

Hands-On Practice 2: Become the Attention Mechanism

Mission: You are deciding which word deserves your attention.
Suppose the word **bank** produces this Query:

$$Q_{\text{bank}} = [1, 0].$$

Three other words produce these Keys:

$$K_{\text{river}} = [1, 0], K_{\text{money}} = [0, 1], K_{\text{the}} = [0, 0].$$

Your Challenge

Calculate the three dot products:

$$Q_{\text{bank}} \cdot K_{\text{river}}, Q_{\text{bank}} \cdot K_{\text{money}}, Q_{\text{bank}} \cdot K_{\text{the}}$$

Then predict:

Which word should “bank” pay the most attention to?

Optional: Ask ChatGPT to check your arithmetic.

Practice 2: Answer — Attention Scores

We have

$$Q_{\text{bank}} = [1, 0].$$

Therefore:

$$Q_{\text{bank}} \cdot K_{\text{river}} = [1, 0] \cdot [1, 0] = 1$$

$$Q_{\text{bank}} \cdot K_{\text{money}} = [1, 0] \cdot [0, 1] = 0$$

$$Q_{\text{bank}} \cdot K_{\text{the}} = [1, 0] \cdot [0, 0] = 0$$

So the relevance scores are:

$$\text{river} = 1, \quad \text{money} = 0, \quad \text{the} = 0$$

Interpretation

The Query from “bank” is more similar to the Key from “river.” Therefore,

bank should pay the most attention to “river.”

This is the basic mechanism behind the deck’s **Query–Key–Value** database analogy.

Visualizing the Attention Matrix

The Attention Weights (Softmax Output)

	The	bank	of	the	river
The	0.90	0.10	0.00	0.00	0.00
bank	0.05	0.30	0.05	0.10	0.50
of	0.00	0.40	0.60	0.00	0.00
the	0.00	0.00	0.00	0.90	0.10
river	0.00	0.45	0.05	0.05	0.45

Notice how "bank" allocates 50% of its attention to "river," completely shifting its mathematical meaning away from "financial institution" and towards "water boundary."

Hands-On Practice 3: Read the Attention Matrix

Mission: Look at this simplified attention matrix:

	dog	chased	cat
dog	0.70	0.20	0.10
chased	0.10	0.20	0.70
cat	0.20	0.60	0.20

Remember:

row = the word doing the attending.

Answer:

- 1 Which word does “chased” pay the most attention to?
- 2 Which word does “cat” pay the most attention to?
- 3 What possible linguistic relationship might this represent?

Researcher Challenge: Ask ChatGPT:

“Give me two different interpretations of this attention matrix. Which interpretation is better supported by the numbers?”

Practice 3: Answer — Reading Attention

Look at each row. For “chased”:

[0.10, 0.20, **0.70**]

Therefore:

chased → cat

For “cat”:

[0.20, **0.60**, 0.20]

Therefore:

cat → chased

Possible Interpretation

The attention pattern suggests that “chased” and “cat” have a strong relationship. One possible interpretation is that the model is tracking **grammatical or semantic relationships**. But we should be careful:

Attention is evidence, not automatically an explanation.

A researcher should do experiments to determine what computation the head is actually performing.

Multi-Head Attention: Multiple Perspectives

A single attention mechanism can only focus on one type of relationship at a time.

- Head 1 might look for grammatical syntax (Adjective $\rightarrow$ Noun).
- Head 2 might track pronouns (He $\rightarrow$ Bob).
- Head 3 might track emotional tone.

Parallel Processing

Modern models like GPT-4 don't use one attention mechanism; they use 96 to 128 **Attention Heads** operating in parallel on every layer, concatenating the results together.

The Feed-Forward Phase ("Memory")

After the words talk to each other in the Attention layer, they pass through a standard Multi-Layer Perceptron (MLP).

- **Attention** is where words share information (routing).
- **Feed-Forward** is where the model retrieves facts and concepts stored in its weights (memory).

The MLP Math

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

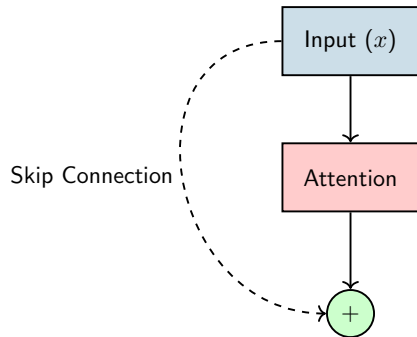
This happens independently for every single word position, allowing massive parallel processing on GPUs.

Residual Connections (Skip Connections)

As networks get extremely deep (100+ layers), signals get lost, and the mathematical gradients "vanish" to zero during training.

The Solution: We add the original input back to the output of the layer.

$$\text{Output} = \text{Layer}(x) + x$$



Hands-On Practice 4: Break the Transformer!

Mission: Discover why Transformers use residual connections. Suppose a layer receives

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

and the layer produces

$$\text{Layer}(x) = \begin{bmatrix} -1 \\ 4 \end{bmatrix}.$$

The Transformer uses:

$$\text{Output} = \text{Layer}(x) + x.$$

Your Challenge: Calculate the output. Then imagine that the layer accidentally produces:

$$\text{Layer}(x) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

What would the residual connection output? Why might it be useful to keep the original information x even after processing it through a complicated layer?

Optional AI experiment: Ask ChatGPT:

"Explain residual connections using an analogy, and why they help deep neural networks."

Practice 4: Answer — Why Skip Connections?

We have

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \quad \text{Layer}(x) = \begin{bmatrix} -1 \\ 4 \end{bmatrix}.$$

Therefore:

$$\text{Output} = \begin{bmatrix} -1 \\ 4 \end{bmatrix} + \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \boxed{\begin{bmatrix} 1 \\ 7 \end{bmatrix}}$$

If the layer produces zero:

$$\text{Layer}(x) = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

then

$$\text{Output} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \boxed{x}.$$

Big Idea: The layer can modify the representation, but the original signal has a direct path through the network to help very deep networks preserve and propagate information.

$$\boxed{\text{Output} = \text{Layer}(x) + x}$$

Summary: One Complete Transformer Block

- ➊ **Input:** Token Embeddings + Positional Encoding.
- ➋ **Self-Attention:** Words query each other for context and update their vectors (QK^T).
- ➌ **Add & Norm:** Residual connection stabilizes the math.
- ➍ **Feed-Forward:** Network applies nonlinear transformations to recall facts/features.
- ➎ **Add & Norm:** Second residual connection.
- ➏ **Repeat:** Pass the output to the next layer (LLMs stack 10 to 100 of these blocks).

The engineering is solved.

But how do we actually interpret the specific patterns these matrices form?

Next: Circuits, Grokking, & Mechanistic Interpretability

Mechanistic Interpretability: Building a Microscope

The Goal

Mechanistic Interpretability aims to reverse-engineer neural networks similar to how biologists reverse-engineer biological systems.

- We want to find the exact algorithms the network has written inside its weights.
- **Analogy:** If the network is a compiled software program, we are trying to read the raw machine code to understand its logic.

The Three Pillars

- ❶ **Features:** The fundamental concepts the network understands.
- ❷ **Circuits:** How the network connects features together to form logic.
- ❸ **Universality:** Do different models learn the exact same circuits?

What is a "Feature"?

A **feature** is a property of the input that a specific neuron (or direction in vector space) responds to.

- **Vision Models:** A neuron might fire only when it sees a curved line, or a dog's snout.
- **Language Models:** A neuron might fire only for French text, or words related to "sadness."

The Problem: Polysemanticity

Because networks have fewer neurons than there are concepts in the universe, they use *superposition*. A single neuron might fire for:

- Pictures of cats.
- The word "bank".
- Base-64 encoded strings.

This makes isolating features incredibly difficult.

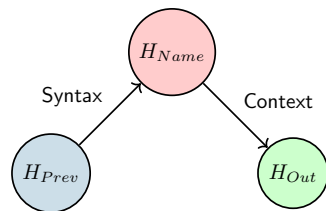
What is a "Circuit"?

If a feature is a variable, a **circuit** is the equation. It is a subgraph of the neural network that performs a specific, human-understandable algorithm.

Example: The "Previous Token" Head

A purely mechanical attention circuit whose only job is to look exactly one word backwards and copy its value.

Input: "The cat sat" → Head focuses entirely on "cat".



A computational subgraph

Anthropic's Discovery: The Induction Head

In 2022, researchers at Anthropic discovered the primary mechanism LLMs use to do "In-Context Learning" (learning from the prompt). They called them **Induction Heads**.

The Algorithm: Prefix Matching

Rule: If token A was followed by token B earlier in the text, and we currently see token A , predict token B .

Example Execution:

- **Prompt:** *"Dr. **Chen** is a Professor at UMass Boston. Dr. **Chen** is tall..."*
- **Step 1:** Find previous instance of "Dr."
- **Step 2:** Look at the token immediately following it ("Chen").
- **Step 3:** Copy "Chen" to the current output.

Hands-On Practice 5: Be an Induction Head

Mission: You are an attention head.

Your algorithm is:

If token A appeared before and was followed by B, predict B when A appears again.

Consider:

The dog likes pizza. The cat likes pizza. The dog likes _____.

Your Algorithm

- 1 Find the current token before the blank.
- 2 Find an earlier occurrence of that token.
- 3 Look at what came immediately after it.
- 4 Copy that token as your prediction.

Your Challenge:

What should the model predict?

Then create your **own sentence** where this algorithm produces an interesting prediction.

Practice 5: Answer — The Induction Head

The relevant part of the prompt is: The dog likes pizza.

Later we see:

The dog likes -----.

The current sequence is:

dog → likes → ?

Earlier, we observed:

dog → likes → pizza

Therefore the induction rule predicts:

pizza

The Circuit

A simplified induction circuit performs:

Find previous A → look at following B → copy B

This is much more interesting than saying “the model predicted pizza.” We have proposed a possible

The Grokking Phenomenon

The Definition

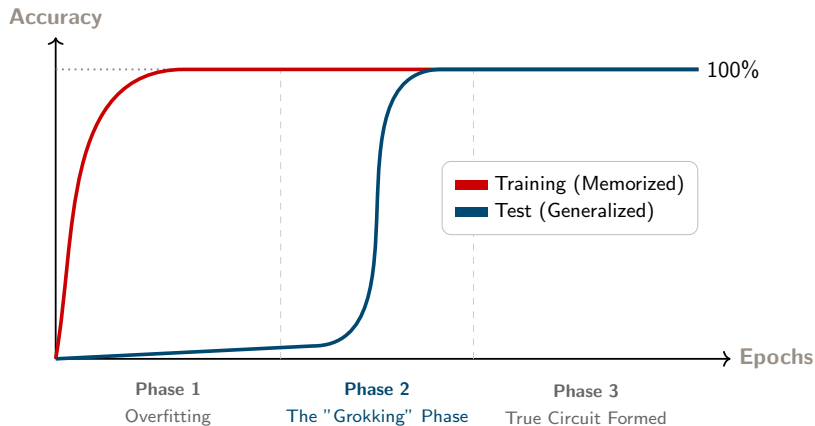
"*Grokking*" occurs when a neural network suddenly generalizes (understands the core rule) long after it has perfectly memorized the training data.

- Discovered by OpenAI when training small models on modular arithmetic (e.g., $a + b \pmod{5}$).
- It completely breaks the classical rules of Machine Learning, which state that once a model overfits (memorizes), training should stop.

Memorization vs. Generalization

- **Memorization:** Creating a giant lookup table in the weights. Fast, but fails on new data.
- **Generalization:** Compiling an elegant algorithm (a circuit). Slow to form, but perfectly accurate on unseen data.

Visualizing Grokking (The Phase Change)



Why Does Grokking Happen?

The Weight Decay Hypothesis

Memorizing a dataset requires messy, massive weight values (a brute-force lookup table).

- We use an optimization trick called **Weight Decay**, which constantly forces the network to make its weights smaller over time.
- A messy lookup table cannot survive weight decay.

The Algorithmic Compression

Eventually, the network "realizes" that compiling a clean, elegant mathematical circuit uses far fewer parameters than maintaining a lookup table.

The sudden spike in accuracy is the exact moment the network deletes the lookup table and activates the algorithmic circuit.

Hands-On Practice 6: Discover the Hidden Rule

Mission: A mysterious AI gives these answers:

Input	Output
$1 + 1$	2
$1 + 2$	3
$2 + 2$	4
$2 + 3$	0
$3 + 3$	1
$3 + 4$	2
$4 + 4$	3

Your Challenge

- 1 What rule is the AI using? Can you give a rule that explains **all** examples?
- 2 Predict: $4+3 =$? $4+2 =$?

Use AI: Ask ChatGPT:

"Find the simplest mathematical rule that explains these examples. Then predict the answer for $4+3$ and $4+2$."

Practice 6: Answer — The Hidden Algorithm

The rule is:

$$a + b \pmod{5}$$

That means:

Add the two numbers, then keep the remainder after dividing by 5.

For example:

$$2 + 3 = 5 \Rightarrow 5 \pmod{5} = 0$$

Therefore:

$$4 + 3 = 7 \Rightarrow \boxed{2}$$

Connection to Grokking. A model could memorize:

$$1 + 1 \rightarrow 2, \quad 1 + 2 \rightarrow 3, \quad \dots$$

like a lookup table. But a model that discovers

$$a + b \pmod{5}$$

has learned an **algorithm** that generalizes to new examples. **That is the key idea behind grokking.**

PhD-Led Lab: Into the Matrix

Lab 1: Attention Heatmaps

- We will use Python to open up a pre-trained Transformer.
- We will feed it a prompt and extract the exact QK^T softmax probability grids.
- You will visualize these matrices as heatmaps to see exactly which words are looking at each other in real-time.

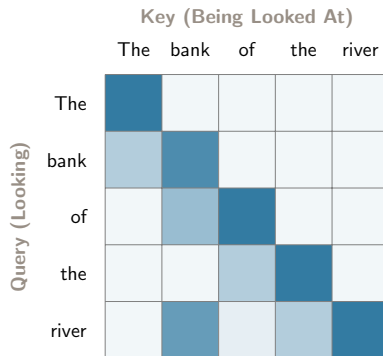
Lab 2: Neuron Exploration

- We will look at a "Tiny Transformer" trained on a toy task.
- You will isolate specific neurons in the MLP layers and attempt to figure out what feature they have learned to activate on.

How to Read an Attention Heatmap

- **Y-Axis:** The word currently doing the "looking" (Query).
- **X-Axis:** The words being looked at (Keys).
- **Color Intensity:** The strength of the attention weight. Darker colors = higher softmax probability.

Notice the strong diagonal line. Most words pay heavy attention to themselves and the immediate previous token.



Thank you!

Questions?

E-mail: inal.mashukov001@umb.edu